

Stream in Java (Java 8)

Stream Hierarchy and Methods

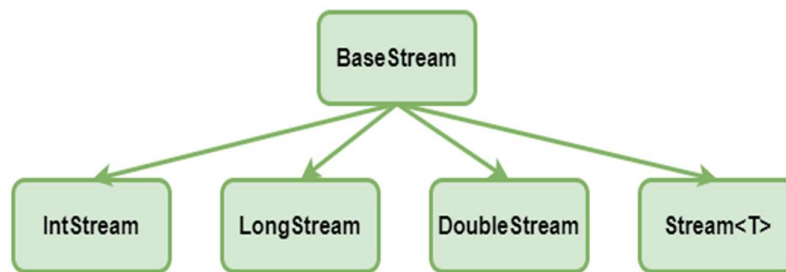
Introduction

- Java Streams were introduced in **Java 8**.
- Stream API provides a **declarative and efficient way** to process collections.
- Streams support operations like **filtering, mapping, sorting, reducing** without modifying original data.

Stream Hierarchy in Java

1 BaseStream Interface (Top Level)

- BaseStream<T, S extends BaseStream<T,S>> is the **parent interface** of all streams.
- Defines core stream control operations.



Important Methods of BaseStream:

- parallel() → Converts stream into parallel stream
- sequential() → Converts parallel stream into sequential
- unordered() → Removes ordering constraints (improves performance)
- close() → Closes the stream (used in I/O streams)

Stream Subinterfaces

Java provides 4 main stream interfaces extending BaseStream:

Stream Type	Used For
Stream<T>	Object/reference types
IntStream	Primitive int values
LongStream	Primitive long values
DoubleStream	Primitive double values

Primitive streams avoid **boxing overhead** and improve performance.

Common Stream Operations

Streams provide a consistent set of operations:

Common Methods

- **filter()** → Selects elements matching condition
- **map()** → Transforms elements
- **distinct()** → Removes duplicates
- **sorted()** → Sorts elements
- **skip(n)** → Skips first n elements
- **findFirst()** → Retrieves first element
- **count()** → Counts elements
- **min()** / **max()** → Finds minimum/maximum
- **reduce()** → Reduces to single value
- **collect()** → Converts stream into collection

Intermediate vs Terminal Operations

Intermediate Operations: Intermediate operations return a new stream and are lazy, meaning they are not executed until a terminal operation is invoked.

Examples: `filter()`, `map()`, `sorted()`, `distinct()`.

Terminal Operations: Terminal operations trigger stream processing and produce a result or side effect. After a terminal operation, the stream cannot be reused.

Examples: `forEach()`, `collect()`, `reduce()`, `count()`.

Key Differences:

Operation Type	Returns Stream?	Examples
Intermediate	Yes	<code>filter()</code> , <code>map()</code> , <code>sorted()</code>
Terminal	No	<code>collect()</code> , <code>forEach()</code> , <code>reduce()</code>

Numeric Stream-Specific Operations

Primitive streams provide extra numeric methods:

- **sum()** → Total sum
- **average()** → Average of elements
- **boxed()** → Converts primitive stream to object stream

Example:

```
int total = IntStream.of(1,2,3).sum();
```

Boxing Primitive Streams

Primitive streams can be converted into object streams using `boxed()`.

Conversions:

- `IntStream` → `Stream<Integer>`
- `LongStream` → `Stream<Long>`
- `DoubleStream` → `Stream<Double>`

Example:

```
Stream<Integer> objStream =  
    IntStream.of(1,2,3).boxed();
```

Used when APIs require wrapper objects instead of primitives.

Examples :

Java Streams provide a modern way to process collections using operations like filtering, mapping, sorting, and aggregation.

1. Sum of Array Elements

Code:

```
import java.util.*;  
  
public class Test {  
    public static void main(String[] args) {  
  
        int arr[] = {30, 20, 10};  
  
        int sum = Arrays.stream(arr).sum();  
  
        System.out.println(sum);  
    }  
}
```

Output:

60

Explanation:

- `Arrays.stream(arr)` creates an **IntStream**
- `sum()` returns total of elements

2. Maximum Element in Array

Code:

```
import java.util.*;

public class Test {
    public static void main(String[] args) {

        int arr[] = {30, 20, 10};

        int max = Arrays.stream(arr).max().getAsInt();

        System.out.println(max);
    }
}
```

Output:

30

Explanation:

- `max()` returns `OptionalInt`
- `getAsInt()` extracts the value

3. Average of Array Elements

Code:

```
import java.util.*;

public class Test {
    public static void main(String[] args) {

        int arr[] = {30, 20, 10};

        double avg =
Arrays.stream(arr).average().getAsDouble();

        System.out.println(avg);
    }
}
```

Output:

20.0

Explanation:

- `average()` returns `OptionalDouble`

4. First N Natural Numbers Containing Digit '5'**Code:**

```
import java.util.stream.*;

public class Test {
    public static void main(String[] args) {

        Stream.iterate(1, x -> x + 1)
            .filter(x -> x.toString().contains("5"))
            .limit(10)
            .forEach(System.out::println);
    }
}
```

Output:

5 15 25 35 45 50 51 52 53 54

Explanation:

- `iterate()` creates infinite stream
- `filter()` selects numbers containing 5
- `limit(10)` takes first 10 results

5. Strings Starting with 'a' and Ending with 'c'**Code:**

```
import java.util.*;

public class Test {
    public static void main(String[] args) {

        String arr[] = {"abc", "cde", "ac", "aec", "xy3"};

        Arrays.stream(arr)
            .filter(x -> x.startsWith("a"))
            .filter(x -> x.endsWith("c"))
            .sorted()
            .forEach(System.out::println);
    }
}
```

```
    }  
}
```

Output:

```
abc  
ac  
aec
```

Explanation:

- Two `filter()` conditions applied
- `sorted()` sorts lexicographically

6. Average Marks of Students**Code:**

```
import java.util.*;  
  
class Student {  
    int marks;  
    Student(int m) { marks = m; }  
    int getMarks() { return marks; }  
}  
  
public class Test {  
    public static void main(String[] args) {  
  
        Student arr[] = {  
            new Student(70),  
            new Student(80),  
            new Student(60)  
        };  
  
        double avg = Arrays.stream(arr)  
                            .mapToInt(x -> x.getMarks())  
                            .average()  
                            .getAsDouble();  
  
        System.out.println(avg);  
    }  
}
```

Output:

```
70.0
```

Explanation:

- `mapToInt()` extracts marks
- `average()` computes mean

7. collect() Method (toMap)

Convert Student Stream into Map

Code:

```
import java.util.*;
import java.util.stream.*;

class Student {
    int roll;
    String name;

    Student(int r, String n) {
        roll = r;
        name = n;
    }

    int getRoll() { return roll; }
    String getName() { return name; }
}

public class Test {
    public static void main(String[] args) {

        Student arr[] = {
            new Student(110, "abc"),
            new Student(101, "bcd"),
            new Student(120, "xyz")
        };

        Map<Integer, String> map =
            Arrays.stream(arr)
                .collect(Collectors.toMap(
                    Student::getRoll,
                    Student::getName
                ));

        System.out.println(map);
    }
}
```

Output:

```
{101=bcd, 110=abc, 120=xyz}
```

8. collect() Method (groupingBy)

Group Students by Marks

Code:

```
import java.util.*;
import java.util.stream.*;

class Student {
    int roll, marks;
    String name;

    Student(int r, String n, int m) {
        roll = r;
        name = n;
        marks = m;
    }

    int getMarks() { return marks; }
}

public class Test {
    public static void main(String[] args) {

        Student arr[] = {
            new Student(110, "abc", 70),
            new Student(101, "bcd", 70),
            new Student(120, "xyz", 60)
        };

        Map<Integer, List<Student>> group =
            Arrays.stream(arr)
                .collect(Collectors.groupingBy(
                    Student::getMarks
                ));

        System.out.println(group);
    }
}
```

Output:

```
70=[Student, Student]
60=[Student]
```

Explanation:

- Groups students based on marks
- Returns Map<Marks, List<Student>>

1 . Intermediate Operations :

Operation	Description	Return Type	Example
<code>filter()</code>	Filters elements based on a predicate.	Stream	<code>stream.filter(n -> n > 10)</code>
<code>map()</code>	Applies a function to each element, transforming it.	Stream	<code>stream.map(String::toUpperCase)</code>
<code>flatMap()</code>	Flattens nested streams into a single stream.	Stream	<code>stream.flatMap(List::stream)</code>
<code>distinct()</code>	Removes duplicate elements.	Stream	<code>stream.distinct()</code>
<code>sorted()</code>	Returns a stream with sorted elements.	Stream	<code>stream.sorted()</code>
<code>limit()</code>	Truncates the stream to a specified size.	Stream	<code>stream.limit(5)</code>
<code>skip()</code>	Skips the first n elements.	Stream	<code>stream.skip(3)</code>
<code>peek()</code>	Performs an action on each element without changing the stream. 	Stream	<code>stream.peek(System.out::println)</code>

2. Terminal Operations :

Operation	Description	Return Type	Example
<code>forEach()</code>	Performs an action for each element.	void	<code>stream.forEach(System.out::println)</code>
<code>toArray()</code>	Collects elements into an array.	Object[]	<code>stream.toArray()</code>
<code>reduce()</code>	Reduces elements to a single value by applying a binary operation.	Optional or Value	<code>stream.reduce(0, Integer::sum)</code>
<code>collect()</code>	Collects elements into a collection (e.g., List, Set).	Collection	<code>stream.collect(Collectors.toList())</code>
<code>min()</code>	Finds the minimum element based on a comparator.	Optional	<code>stream.min(Comparator.naturalOrder())</code>
<code>max()</code>	Finds the maximum element based on a comparator.	Optional	<code>stream.max(Comparator.naturalOrder())</code>

<code>count()</code>	Counts the number of elements in the stream.	long	<code>stream.count()</code>
<code>anyMatch()</code>	Returns true if any element matches the predicate.	boolean	<code>stream.anyMatch(n -> n > 10)</code>
<code>allMatch()</code>	Returns true if all elements match the predicate.	boolean	<code>stream.allMatch(n -> n > 0)</code>
<code>noneMatch()</code>	Returns true if no elements match the predicate.	boolean	<code>stream.noneMatch(n -> n < 0)</code>
<code>findFirst()</code>	Returns the first element, if present.	Optional	<code>stream.findFirst()</code>
<code>findAny()</code>	Returns any element, especially useful in parallel streams.	Optional	<code>stream.findAny()</code>

Key Differences Between Intermediate and Terminal Operations

Aspect	Intermediate Operations	Terminal Operations
Purpose	Transform a stream into another stream.	Produce a result or a side effect.
Execution	Lazy: evaluated only when a terminal operation is invoked.	Eager: executes immediately and consumes the stream.
Return Type	Stream (allows further chaining).	Non-stream result (e.g., <code>List</code> , <code>int</code> , <code>boolean</code> , <code>void</code>).
Can Be Chained	Yes, can be used in a pipeline.	No, terminates the stream pipeline.
Side Effects	Typically avoids side effects.	May cause side effects (e.g., printing, collecting data).
Examples	<code>filter()</code> , <code>map()</code> , <code>sorted()</code> , <code>distinct()</code> , <code>peek()</code> .	<code>forEach()</code> , <code>collect()</code> , <code>reduce()</code> , <code>count()</code> , <code>findFirst()</code> .